

Programmation Objet Avancée

Cours 1

Ahmed ADJIBADE

ahmed.adjibade@bivariant.com

Année universitaire : 2026-2027

Ce qu'il faut installer



Outil	Rôle
JDK 25	Compiler et exécuter du Java
Git, Github CLI	Versionner le code, Se connecter depuis le terminal
VS Code ou IntelliJ IDEA	Écrire et lancer le code

Maven ne s'installe pas. Le dépôt fournit le Maven Wrapper → `./mvnw`

Un compte GitHub est nécessaire

Installation des outils

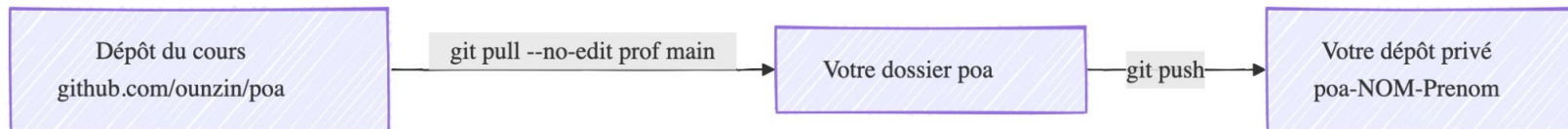
Tout est décrit dans INSTALLATION.md, sur <https://github.com/ounzin/poa>



Attention : `./mvnw -v` doit afficher Java 25. Sinon, Maven utilise un autre JDK...

Deux dépôts, deux sens

Tout est décrit dans INSTALLATION.md, sur <https://github.com/ounzin/poa>



prof : on y reçoit les supports, les exercices et les TP.

origin : on y sauvegarde son travail

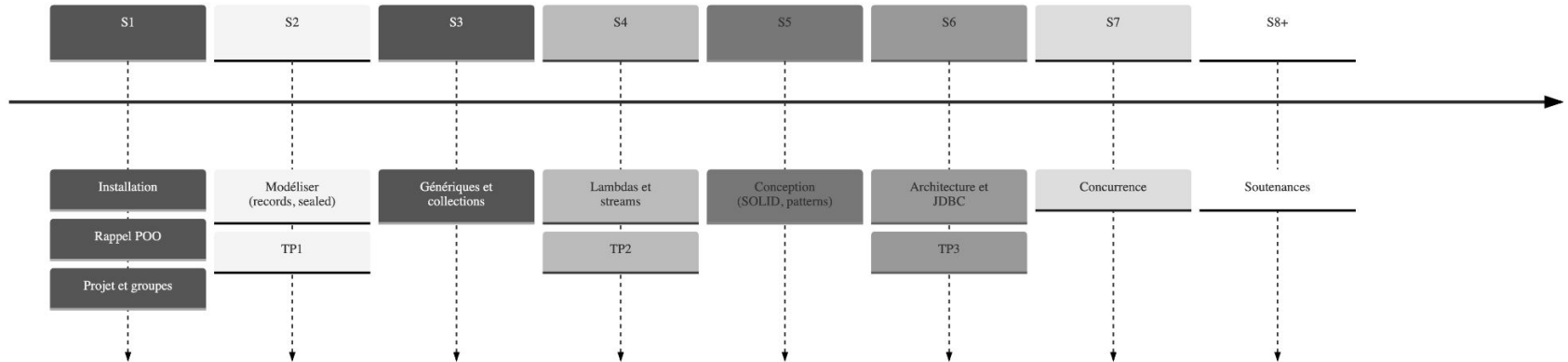
src/ est à vous ... Ne modifiez aucun autre fichier

Le détails des commandes est dans [README.md](#)

Programme

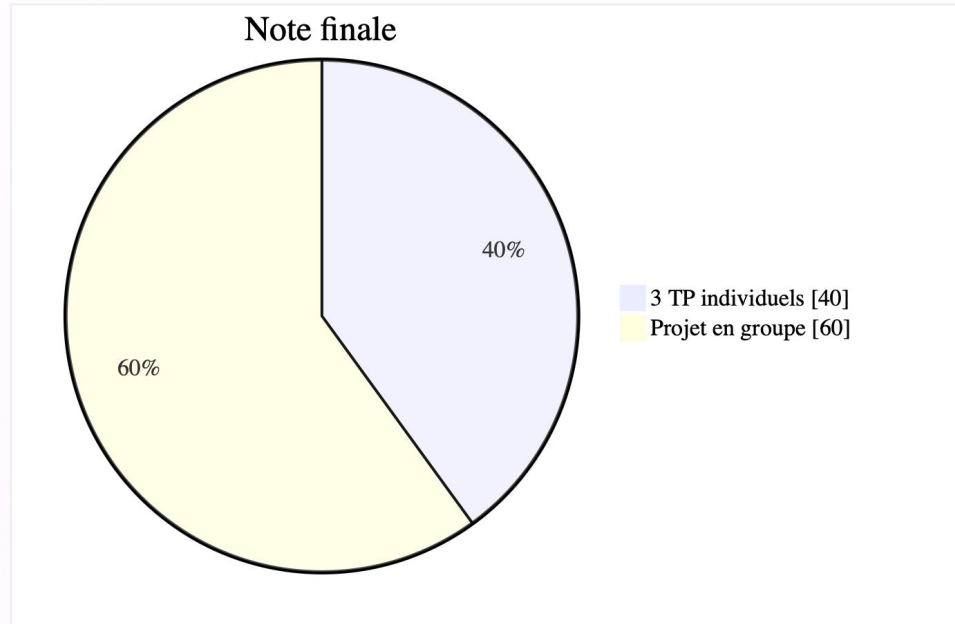
7 séances de contenu, 3 TP, 1 projet présenté dès la séance 8.

Les 8 séances



L'évaluation

3 TP individuels (40 %) et un projet en groupe (60 %).



Élément	Où
Énoncé	tp/tpX/enonce.md
Tests fournis	tp/tpX/tests/
Votre code	src/main/java/fr/poa/tpX

Instructions pour le rendu:

- Un zip par TP, nommé ***TPx_NOM_Prenom_NumEtudiant.zip***, envoyé par mail avant le début de la séance 8.
- Respectez à la lettre les noms de classes et de méthodes de l'énoncé. La correction ajoute d'autres tests ... si votre code empêche la compilation, la note de l'exercice est 0.
- La création du zip est décrite dans [README.md](#) section "Les TP"

Programmation Objet Avancée ... Quèsaco ?

La POO organise un programme en objets, qui regroupent des données et les opérations sur ces données.

“Avancée” signifie concevoir des modèles robustes et faciles à faire évoluer avec le Java Moderne



Programmation Objet Avancée ... Quèsaco ?

Bases (déjà vues ...)	Ce cours
Classes, héritage, méthodes	Modéliser précisément : records, types scellés
Tableaux	Généricité et collections
Boucles	Lambdas et streams
Du code qui fonctionne	Du code testé et conçu pour évoluer : SOLID, patterns
Un seul fil d'exécution	Concurrence

Classe et objet

Une classe est un modèle, autrement dit un type. Un objet est une instance de ce modèle, créée avec *new*. Il possède :

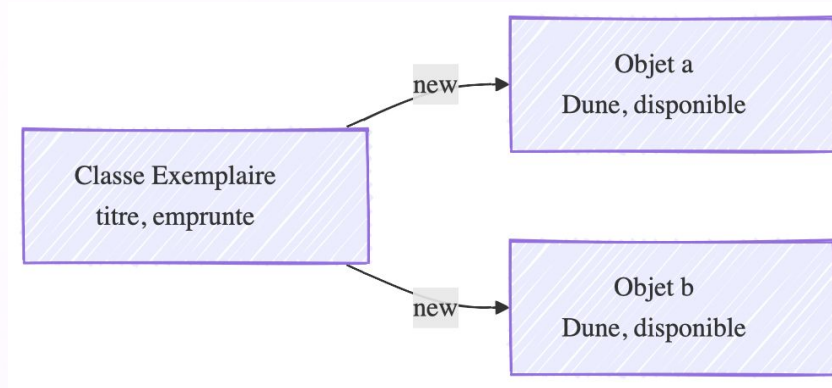
- Un **état** : ses attributs
- Un **comportement** : ses méthodes
- Une **identité** : il reste distinct de tout autre objet

```
Exemplaire a = new Exemplaire("Dune");
```

```
Exemplaire b = new Exemplaire("Dune");
```

```
boolean memeObjet = (a == b); // false : deux objets distincts, même état
```

Classe et objet



Attention :

- `==` compare les identités, pas les états.
- `equals` ne compare les états que si la classe le redéfinit. Sinon, il se comporte comme `==` : ici, `a.equals(b)` vaut aussi `false` ...
- On redéfinira `equals` en séance 2

Les 4 piliers de la POO

Pilier	Idée	Dans la billetterie
Encapsulation	L'état est caché. On passe par des méthodes qui protègent les règles	Une place ne se vend que par <i>emprunter()</i>
Abstraction	On manipule ce qu'un objet fait, pas la façon dont il le fait	<i>Emprutable</i> : "a une durée d'emprunt"
Héritage	Une classe en spécialise une autre : relation "est un"	Un livre est un document
Polymorphisme	Un même appel produit un comportement qui dépend de l'objet réel	<i>document.dureeEmprunt()</i>

Une classe : un état privé, des méthodes publiques

```
package fr.poa.cours01;

public class Exempleaire {

    private final String titre;

    private boolean emprunte;

    public Exempleaire(String titre) {

        this.titre = titre;

    }

    public boolean estDisponible() {

        return !emprunte;

    }

}
```

- L'état est *private*
- Un attribut *final* est fixé une fois pour toutes dans les constructeur
- Un attribut non initialisé prend une valeur par défaut: *false*, *0* ou *null*.

Encapsuler c'est protéger les règles

```
public Exempleire(String titre) {  
    if (titre == null || titre.isBlank()) {  
        throw new IllegalArgumentException("Titre obligatoire");  
    }  
    this.titre = titre;  
}  
  
public void emprunter() {  
    if (emprunte) {  
        throw new IllegalStateException("Déjà emprunté : " + titre);  
    }  
    emprunte = true;  
}
```

- L'objet garantit ses propres règles : impossible de créer un exemplaire sans titre, ou de l'emprunter deux fois.
- *IllegalArgumentException* signale un argument invalide.
- *IllegalStateException* signale une opération que l'état de l'objet interdit.

Attention : un *setEmprunte(boolean)* public détruirait cette garantie. Pas de setter par réflexe.

Un règle métier = un test

```
package fr.poa.cours01;

import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.jupiter.api.Test;

class ExempleaireTest {

    @Test

    void impossible_d_emprunter_deux_fois() {

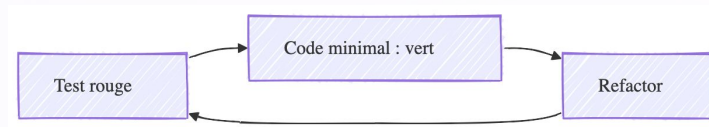
        Exempleaire exempleaire = new Exempleaire("Dune");

        exempleaire.emprunter();

        assertThrows(IllegalStateException.class, () -> exempleaire.emprunter());

    }

}
```



- Le test exprime la règle en code.
- La syntaxe `() -> ...` signifie “le code à exécuter”

Classe abstraite : factoriser l'état commun

```
package fr.poa.cours01;

public abstract class Document {
    private final String titre;

    protected Document(String titre) {
        this.titre = titre;
    }

    public String titre() { return titre; }

    public abstract int dureeEmprunt(); // en jours
}
```

- Une classe *abstract* ne s'instancie pas. Elle oblige chaque sous-classe à écrire *dureeEmprunt()*
- Un constructeur *protected* n'est appelé que par les sous-classes

Fichiers : *Livre.java* et *Dvd.java*

```
package fr.poa.cours01;

public class Livre extends Document {

    public Livre(String titre) {

        super(titre);

    }

    @Override

    public int dureeEmprunt() { return 21; }

}
```

Fichiers : *Livre.java* et *Dvd.java*

```
package fr.poa.cours01;

public class Dvd extends Document {

    public Dvd(String titre) {

        super(titre);

    }

    @Override

    public int dureeEmprunt() { return 7; }

}
```

- *super(...)* appelle le constructeur parent.
- *@Override* fait vérifier par le compilateur que la méthode redéfinie existe bien.

Polymorphisme : même appel, comportements différents

Fichiers : *DocumentTest.java*

```
package fr.poa.cours01;

import static org.junit.jupiter.api.Assertions.assertEquals;
import java.util.List;
import org.junit.jupiter.api.Test;

class DocumentTest {

    @Test
    void la_duree_depend_du_type_de_document() {

        List<Document> documents = List.of(new Livre("Dune"), new Dvd("Alien"));

        int total = 0;

        for (Document document : documents) {

            total += document.dureeEmprunt();

        }

        assertEquals(28, total);

    }

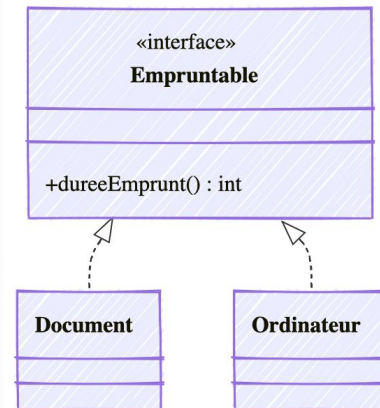
}
```

On manipule le type général, Document. À l'exécution, c'est l'objet réel qui choisit la méthode.

Interface : un contrat

Fichiers : *Empruntable.java* , *Document.java* , & *Ordinateur.java*

```
public interface Empruntable {  
    int dureeEmprunt(); // en jours  
}  
  
public abstract class Document implements Empruntable { /* inchangé */ }  
  
public class Ordinateur implements Empruntable {  
    @Override  
    public int dureeEmprunt() { return 3; }  
}
```



Une interface dit ce qu'un objet sait faire, sans porter d'état. Des classes sans lien de parenté peuvent la respecter.

Classe abstraite ou interface ?

	Classe abstraite	Interface
Contient de l'état	Oui	Non
Une classe peut en avoir	Une seule (<i>extends</i>)	Plusieurs (<i>implements</i>)
Sert à	Factoriser du code commun	Définir un contrat

Par défaut, une interface. Une classe abstraite quand des sous-classes partagent de l'état.

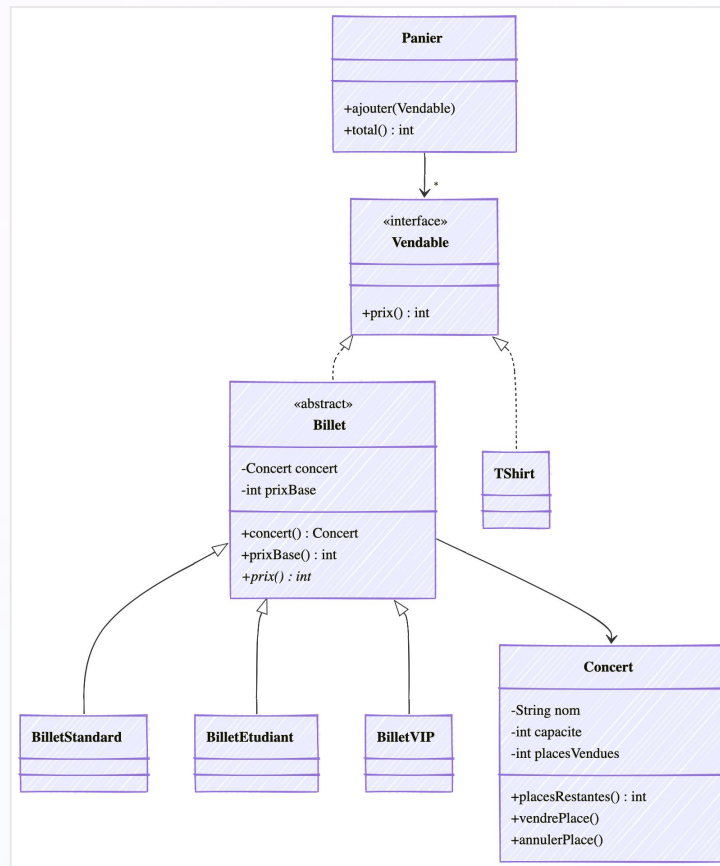
Le fil rouge des exercices : une billetterie de concerts

Par hypothèse :

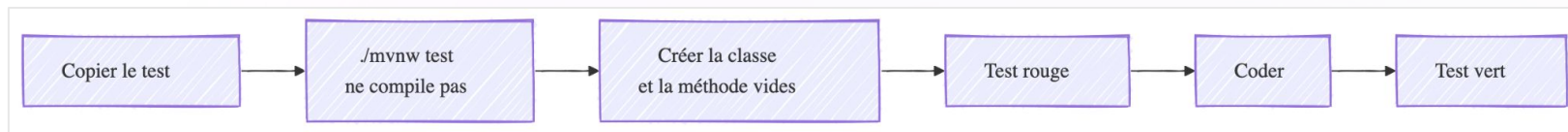
- Un concert a une capacité limitée.
- Un billet étudiant coûte 30 % de moins qu'un billet standard.
- Un billet VIP coûte 1,5 fois le prix de base.
- La billetterie vend aussi des t-shirts.

Attention

- L'argent se compte en centimes (*int*), jamais en double : *0.1 + 0.2* vaut *0.300000000000000004*.
- Entre deux centimes, on arrondit au centime inférieur.



Ouvrez [exercices/cours-01/enonce.md](#) dans votre dépôt. Pour chaque question :



Un test qui ne compile pas est un test rouge. On commence par créer la classe et la méthode, vides.

En fin de séance, sauvegardez votre travail : voir [README.md](#), section « Routine d'un cours ».